RL-Based Smart Elevator Scheduling with a Factorised Double DQN

J. Arjun Naik (B23CM1018)
GitHub: `https://github.com/Mico1704/RL-Based-Smart-elevator-scheduling`

1 Problem and environment

A central agent controls 3 cars in a 10-floor building (capacity 8) to cut passenger waiting time and energy. Our custom Gymnasium simulator draws Poisson arrivals with pattern-specific rates and origin/destination mixes (up-peak, down-peak, lunch, quiet); an episode is 200 ticks. One tick moves a car one floor or opens its doors (unload, then board FIFO). Energy per car-tick: 1 per floor moved, +0.5 per start, +0.5 per stop; idling is free.

2 RL formulation

State (R^{137}). Per car: one-hot floor (10), direction, load, in-car buttons (10), and 12 relational features (pickup/drop target here, nearest below/above with distance, whether another car is closer). Building: per floor up/down call counts and oldest-call age (30), traffic-pattern one-hot (4), time fraction (1).
Action. Per car {down, stop, up}; joint space $3^3 = 27$.
Reward. $r_t = -\kappa\,(N_t^{wait} + \beta N_t^{ride}) - \lambda E_t$ with $\kappa = 0.1$, $\beta = 0.25$, $\lambda = 0.1$; E_t is the step's energy.
Algorithm. Double DQN written in NumPy, with a value-decomposed head $Q(s, \mathbf{a}) = \sum_{i=1}^{3} Q_i(s, a_i)$ so the 27-way argmax splits into three. MLP 128-128, Adam (3e-4), 8-step returns, $\gamma = 0.95$, soft target updates, 250k steps on mixed traffic. Plain one-step DQN collapsed to idling (it "saved" energy by never boarding anyone); n-step returns, relational features and *guided exploration* (30–80% of early episodes driven by a noisy Nearest-car/SCAN controller, learning only from reward) fixed this.

3 Baselines and evaluation

Random, Nearest-car (each call to the closest car with room) and SCAN-style (sweep and reverse). All controllers use the same 20 unseen seeds per pattern. Waiting time counts still-queued passengers' current age.

4 Results and discussion

The agent does not meet the success criterion. It learned to serve most passengers (vs. none for the first version) and uses less energy than Nearest-car in 12–19 of 20 seeds per pattern, but its waiting time is 3–6× higher (better on only 1 of 80 paired runs), with 5–38 passengers unserved per episode; part of the energy saving comes from that unserved demand. Nearest-car is a strong baseline in this small building. λ **sweep** (DQN, averaged over patterns; Nearest-car in brackets): $\lambda = 0.02$: wait 21.2, energy 377; $\lambda = 0.1$: 18.5, 290; $\lambda = 0.3$: 35.7, 160 (Nearest-car 3.5, 342). Raising λ cuts energy but lengthens waits, as designed; no setting reaches Nearest-car's waiting time. Likely causes: sparse credit across a shared team reward, 250k steps, and limited tuning. Longer training, a larger network and per-car credit assignment are future work.

Pattern	Controller	Avg wait	Max wait	Energy
up-peak	Random	47.4±9.4	128.1±20.3	555.3±22.3
up-peak	Nearest-car	3.9±0.8	17.2±4.1	355.8±29.8
up-peak	SCAN-style	4.2±0.8	15.8±3.9	532.5±20.1
up-peak	DQN	25.3±15.2	90.7±39.3	272.6±88.4
down-peak	Random	37.6±9.9	151.7±20.8	557.7±17.5
down-peak	Nearest-car	4.1±0.8	25.1±10.8	406.2±34.8
down-peak	SCAN-style	6.4±0.9	22.3±3.1	575.5±11.1
down-peak	DQN	19.4±13.4	94.2±43.9	370.1±84.9
lunch	Random	34.7±5.9	137.3±22.5	554.6±16.3
lunch	Nearest-car	3.2±0.3	15.8±4.1	465.3±32.1
lunch	SCAN-style	5.4±1.2	22.1±4.7	567.8±10.1
lunch	DQN	12.0±7.2	61.2±32.0	381.1±60.8
quiet	Random	18.9±8.2	70.2±29.0	552.7±24.4
quiet	Nearest-car	2.7±0.5	7.4±2.5	139.7±32.7
quiet	SCAN-style	3.1±0.5	8.3±2.8	259.2±47.5
quiet	DQN	17.3±9.1	81.0±36.5	134.9±42.5

Table 1: Mean ± std over 20 seeds (time in ticks).

References

[1] Sutton & Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.

[2] van Hasselt et al., Deep RL with Double Q-learning, AAAI 2016.

[3] Sunehag et al., Value-Decomposition Networks, 2017.

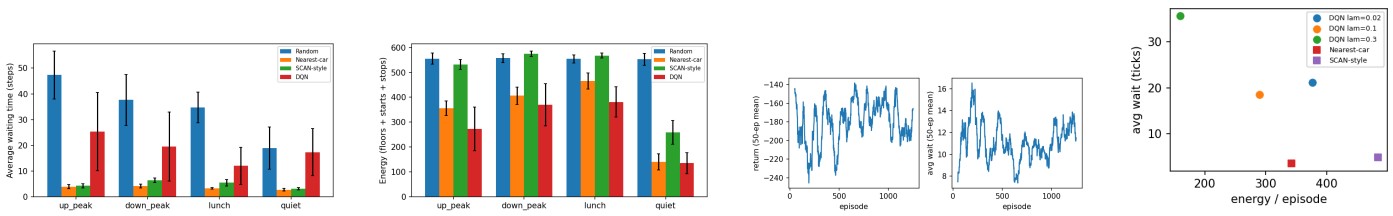


Figure 1: Average wait, energy per episode, DQN training curve, and λ trade-off (mean over all patterns, 20 seeds each).